SHERLOCK

Security Review For Prodigy Finance



Collaborative Audit Prepared For: **Prodigy Finance**
Lead Security Expert(s): **deth**
Obsidian
Date Audited: **September 29 - October 8, 2025**

Introduction

Prodigy.Fi is a fully onchain platform for Dual Currency Investment (DCI) vaults, offering fixed yields in any market through a permissionless, non-custodial vault marketplace. With Prodigy.Fi, users can lock in guaranteed returns at subscription, accumulate BTC or ETH at target prices, and put idle stablecoins to work with passive yield strategies.

Scope

Repository: prodigyfi/brt-dci-contracts

Audited Commit: b65f93ce1d1635768d73d26e4ab216b3d67edc12

Final Commit: 5eae710c669e578661f554be69d3b7a27624e27c

Files:

- src/CollateralPoolV2.sol
- src/FactoryCore.sol
- src/Factory.sol
- src/libraries/CalcLib.sol
- src/libraries/StructsLib.sol
- src/Router.sol
- src/VaultBatchManager.sol
- src/VaultCore.sol
- src/Vault.sol
- src/VaultV2.sol

Final Commit Hash

5eae710c669e578661f554be69d3b7a27624e27c

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.

- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	2	3

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue M-1: Since CollateralPoolV2 does not support partial withdrawals, minor LP shortfalls can fully block user withdrawals [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2025-09-prodigy-finance-sept-25th/issues/12

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The CollateralPoolV2 contract does not support partial withdrawals, causing full withdrawal attempts to revert even when only a small portion of the amount is unavailable.

Vulnerability Detail

If an LP in CollateralPoolV2 has almost, but not entirely enough funds, to cover a user's withdrawal, the transaction will revert.

This occurs because `_updateLpBalance` enforces a strict require check that the LP's `ownDeposit` must meet the withdrawal amount. As a result, users with large positions may be unable to withdraw any funds when there is only a small shortfall.

This issue can only occur with whitelisted LP's, making it a very unlikely scenario.

Impact

A minor shortfalll from the LP will completely block user withdrawals, leaving user funds inaccessible until the LP replenishes the missing balance, if the LP does not fill the shortfall, users will permanently lose funds.

Code Snippet

https://github.com/sherlock-audit/2025-09-prodigy-finance-sept-25th/blob/3ce9fb38cffe4a26e601e2d556b56b201a2a513b/brt-dci-contracts/src/CollateralPoolV2.sol#L277-L283

Proof of Concept

Add the following file to `brt-dci-contracts/test/POC.t.sol`

The test `test_Shortfall_POC` shows that a withdrawal of over 250 USDC is blocked by a tiny shortfall of 0.1 USDC

CODE REDACTED

CODE REDACTED

CODE REDACTED

CODE REDACTED

CODE REDACTED

CODE REDACTED

CODE REDACTED

Run the test with the command `forge test --mt test_Shortfall_POC -vv --via-ir`

Console Output:

CODE REDACTED

Tool Used

Manual Review

Recommendation

Consider adding support for partial withdrawals so users can withdraw the available portion while tracking any remaining owed amount.

Discussion

shawnwang0715

Thanks for pointing this out. Since we are the only whitelisted LP, we can guarantee that we will have enough tokens to fill the shortfall. We won't need to fix this issue now.

lpetroulakis

Issue acknowledged by the team. Won't be fixed at this time.

Issue M-2: Owner can front-run depositors by calling `adjustYieldValue` and setting it to dust amounts [RESOLVED]

Source: https://github.com/sherlock-audit/2025-09-prodigy-finance-sept-25th/issues/14

Summary

Owners front-run depositors by setting the `yieldValue` to dust amounts, so users get no yield and no trading fee is enacted on the owner for the deposit as well.

Vulnerability Detail

The owner can front-run any depositor by setting `yieldValue` to a dust amount, this way the depositor will not get any yield after he deposits and also no trading fee will be enacted on the owner.

He can then back-run the deposit, setting the value back to the original one. The contract are planned to be deployed on mainnet where MEV style attacks are easily doable. While users may lose confidence in the owner, they have to wait for the option to finish in order to get their funds back.

Impact

Frozen funds, as users will deposit for the yield, this way they are just freezing their funds in the contract until `VaultCore::execute` is called and the Vault is closed.

Code Snippet

https://github.com/sherlock-audit/2025-09-prodigy-finance-sept-25th/blob/3ce9fb38cffe4a26e601e2d556b56b201a2a513b/brt-dci-contracts/src/Vault.sol#L145

Tool Used

Manual Review

Recommendation

We recommend adding a `minYieldValue` for depositors, so they can choose what minimum yield they would accept when depositing. Another fix can be adding a

timelock to `adjustYieldValue`, where changes to `yieldValue` take effect after X number of blocks/seconds/hours etc.

Issue L-1: Whitelisted LPs bypass balance checks, exposing users to withdrawal shortfall risk [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2025-09-prodigy-finance-sept-25th/issues/11

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Whitelisted LPs bypass balance checks during deposits

Vulnerability Detail

When an LP is whitelisted, the `processUserDeposit` function skips the balance verification checks, and assumes the LP will later cover any shortfall before withdrawals post-expiry.

Impact

If the whitelisted LP is unable to cover a shortfall, users who deposited will be unable to withdraw their balances.

Code Snippet

https://github.com/sherlock-audit/2025-09-prodigy-finance-sept-25th/blob/3ce9fb38cffe4a26e601e2d556b56b201a2a513b/brt-dci-contracts/src/CollateralPoolV2.sol#L177-L184

Tool Used

Manual Review

Recommendation

Consider always performing balance checks, even for whitelisted LPs.

Otherwise, consider documenting this behaviour so depositors are aware of the trust assumptions associated with whitelisted lp's/vault owners.

Discussion

shawnwang0715

Thanks for pointing this out. Since we are the only whitelisted LP, we can guarantee that we will have enough tokens to fill the shortfall. We won't need to fix this issue now.

lpetroulakis

Issue acknowledged by the team. Won't be fixed at this time.

Issue L-2: `_transferVaultCreationFunds` has a leaky check for `requiredETH` [RESOLVED]

Source: https://github.com/sherlock-audit/2025-09-prodigy-finance-sept-25th/issues/13

Description Using `msg.value` here is not correct 100% of the time, in the case where the aggregator.getPrice uses getLatestEmaPrice, the `PythAggregator` will attempt to updatePriceFeeds and refund any excess back to the Factory.

When we get to `msg.value < requiredETH`, `msg.value` no longer represents the correct value, as some of `msg.value` was used for Pyth, thus the check can pass when it shouldn't.

Impact No security impact, as if there aren't enough funds in the contract `weth.deposit` will revert, but the check is technically wrong

Line ref https://github.com/sherlock-audit/2025-09-prodigy-finance-sept-25th/blob/3ce9fb38cffe4a26e601e2d556b56b201a2a513b/brt-dci-contracts/src/Factory.sol#L262

Recommendation Use `address(this).balance` instead or cache the original `msg.value` and decrement it if Pyth used some of it

Issue L-3: Missing chain ID in signed message hash enables cross-chain replay [RESOLVED]

Source: https://github.com/sherlock-audit/2025-09-prodigy-finance-sept-25th/issues/15

Summary

The message hash for signed deposits in VaultV2 is calculated in the following way:

CODE REDACTED

Since this does not include the chainId in the signature, the signatures can be re-used across chains, as long as the vault is deployed at the same address (and the expected signer is the same).

Vulnerability Detail

Impact

In certain cases, signed deposits can be made on chains where the signature was not intended to be used. This allows the `yieldValue` to be changed to a potentially unintended value.

Code Snippet

https://github.com/sherlock-audit/2025-09-prodigy-finance-sept-25th/blob/3ce9fb38cffe4a26e601e2d556b56b201a2a513b/brt-dci-contracts/src/VaultV2.sol#L102-L105

Tool Used

Manual Review

Recommendation

Consider including `block.chainid` in the message to be signed

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.